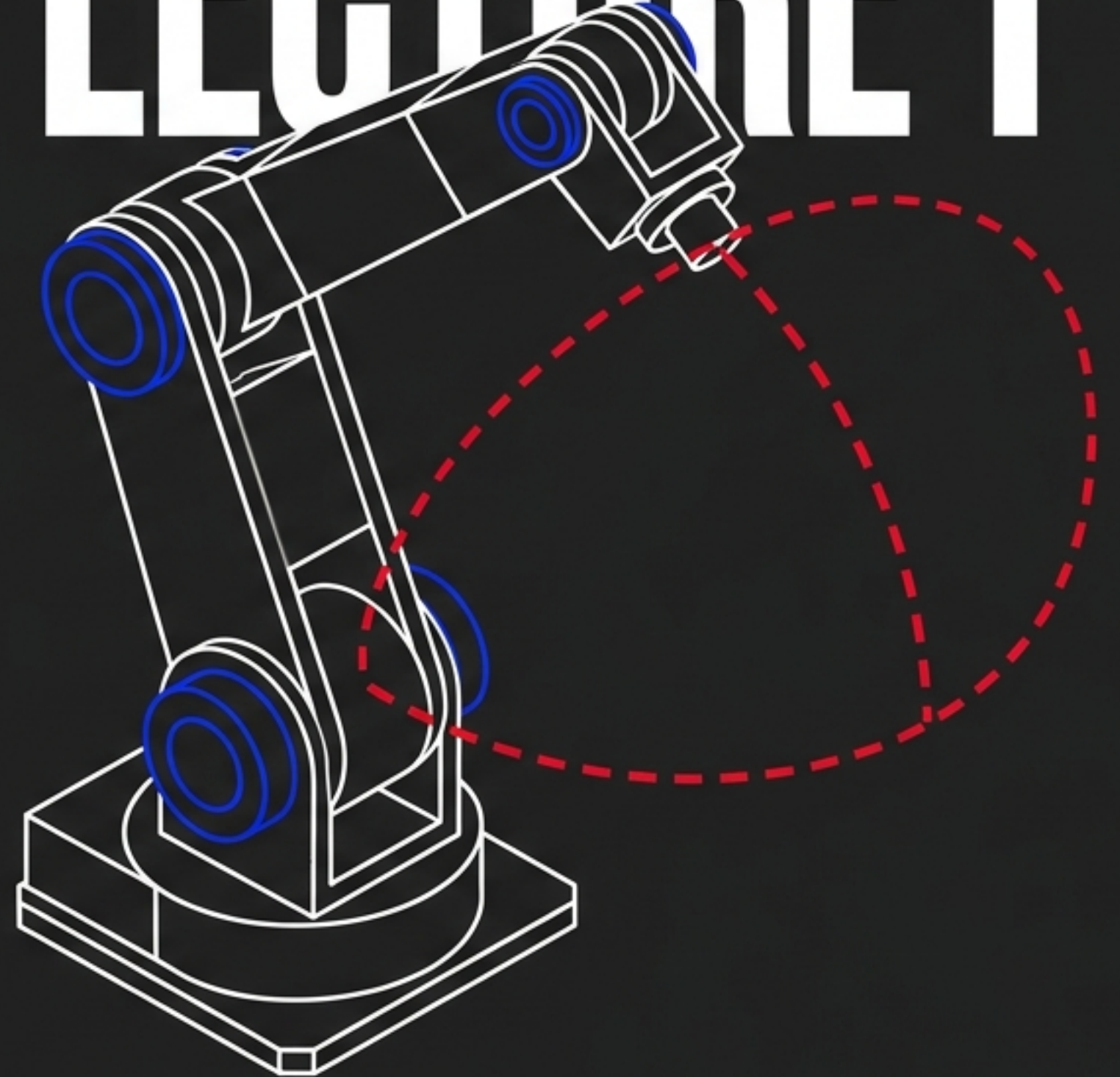


ROBOTICS • LECTURE 1

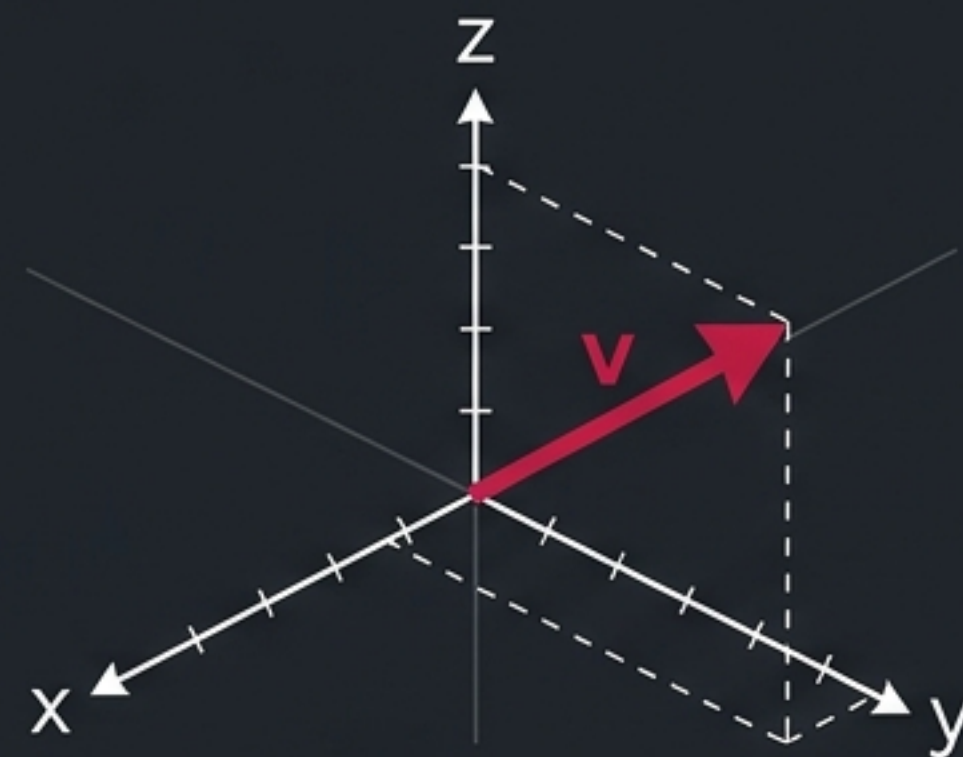
Kinematics & The Math of Motion

Translating joint commands
into real-world coordinates.



01 THE CANVAS

Vectors & Linear Space



INDEPENDENT (Basis)



$$c_1v_1 + c_2v_2 + \dots + c_nv_n \neq 0$$

DEPENDENT (Coplanar)

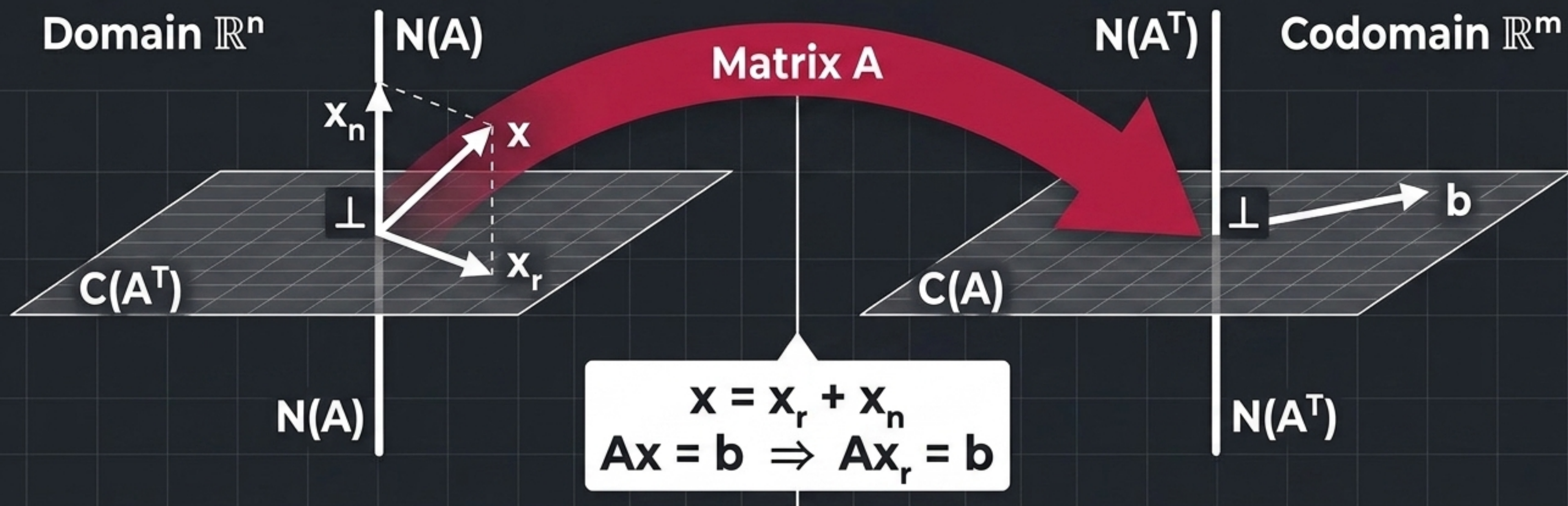


Vectors collapse into a lower dimension.
No new span is added.

**A basis requires independent vectors that span the entire space.
The number of these vectors defines the DIMENSION.**

02 THE BRIDGE

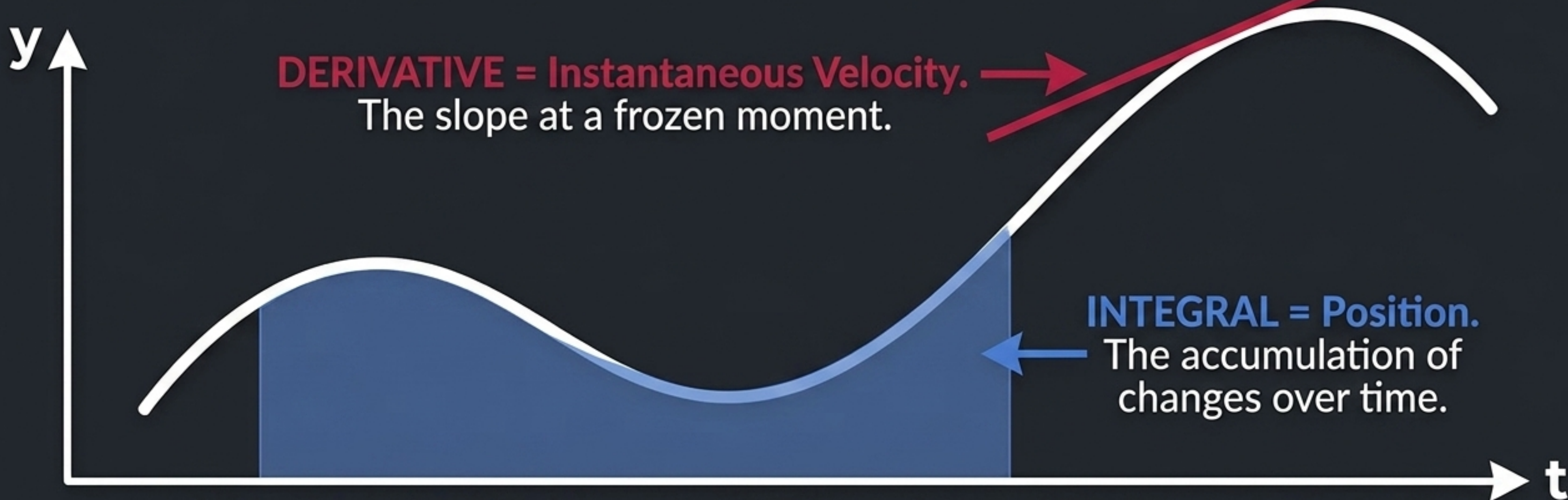
The 4 Fundamental Subspaces



Matrix A acts as an engine, transforming orthogonal axes of the row space into the column space, while crushing the null space to zero

03 CONTINUOUS MOTION

Calculus 101: Rates & Accumulation



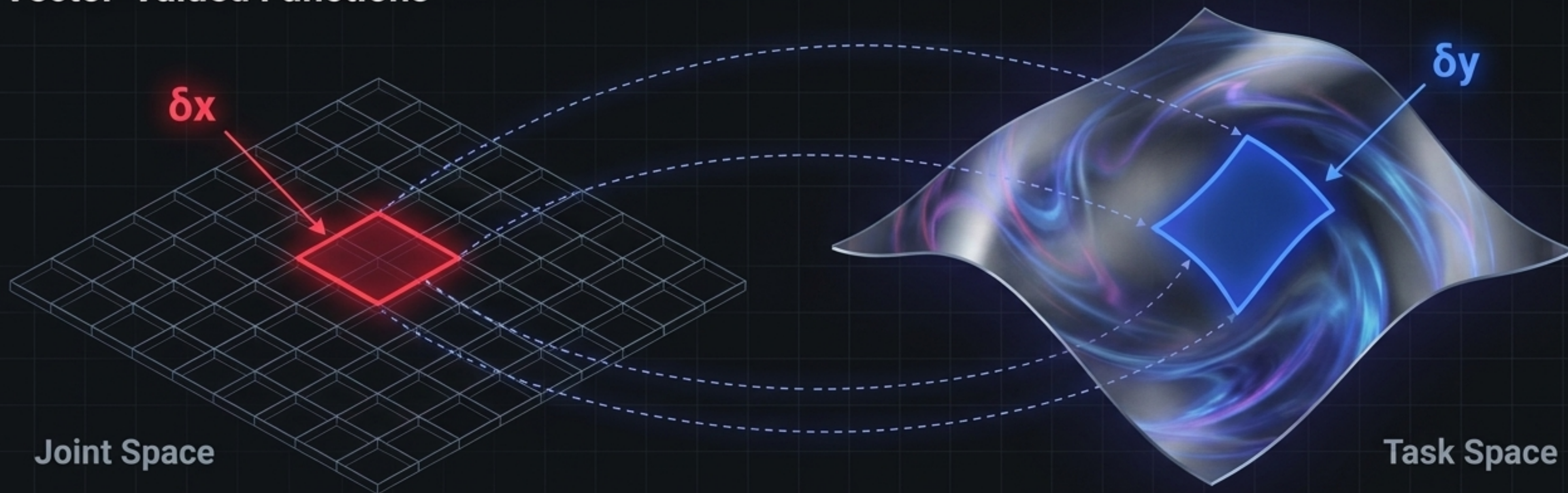
VELOCITY: $v_p \neq \dot{r}_{AP}$

(unless differentiating in a non-moving inertial frame).

ACCELERATION: $a_p = \dot{v}_p$

04 INFINITESIMAL CHANGES

Vector-Valued Functions

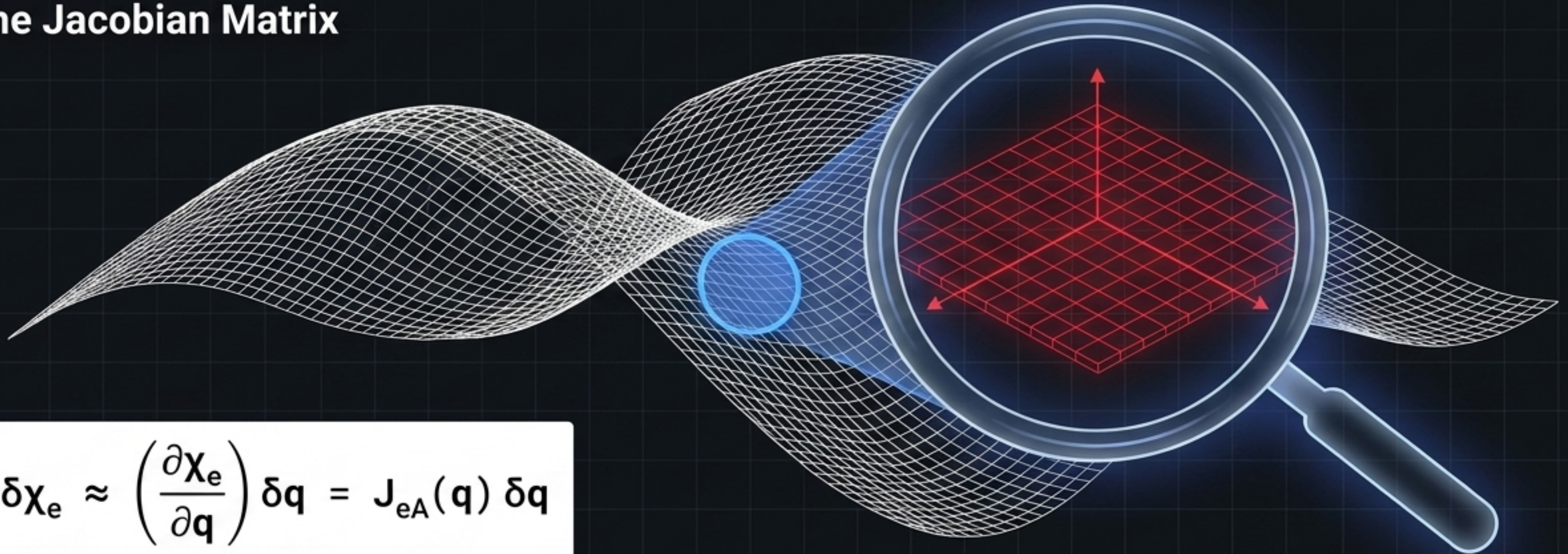


What happens when a function outputs a vector? A tiny nudge in the input joint space (δx) causes a proportional, but geometrically transformed, nudge in the output task space (δy).

$$\delta y \approx (\text{Linear Transformation}) \cdot \delta x$$

05 THE MAGNIFYING GLASS

The Jacobian Matrix



$$\delta \chi_e \approx \left(\frac{\partial \chi_e}{\partial \mathbf{q}} \right) \delta \mathbf{q} = \mathbf{J}_{eA}(\mathbf{q}) \delta \mathbf{q}$$

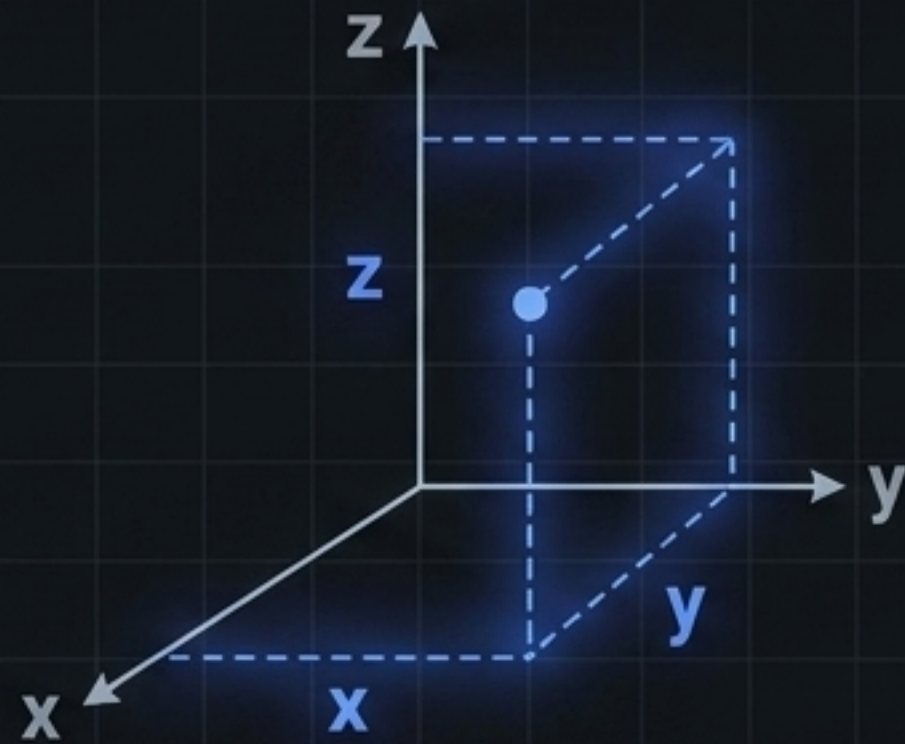
The Jacobian $\mathbf{J}_{eA}(\mathbf{q})$ is the multi-dimensional derivative. It represents the best linear approximation of a non-linear function near a given point. It maps joint velocities to end-effector velocities.

$$\dot{\chi}_e = \mathbf{J}_{eA}(\mathbf{q}) \dot{\mathbf{q}}$$

06 TRACKING POINTS

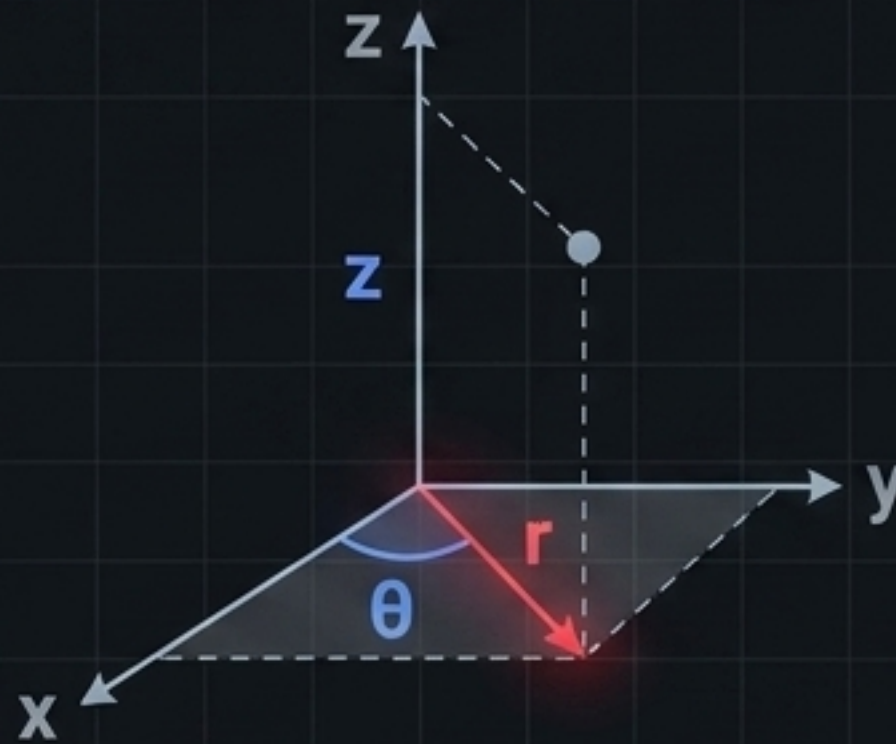
Representations of Position

CARTESIAN (x, y, z)



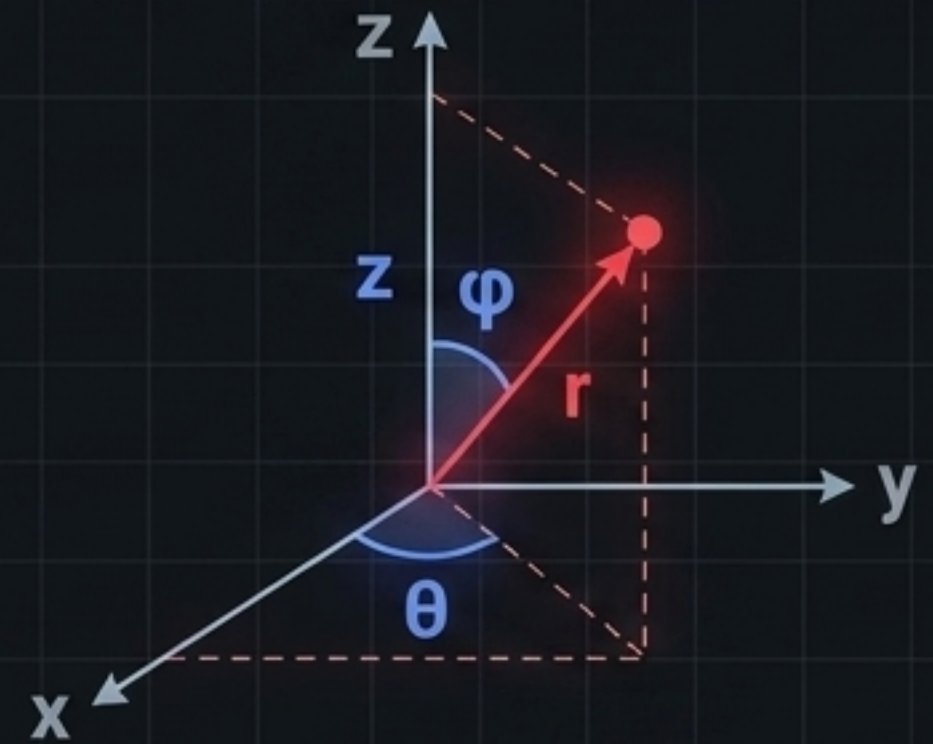
$$\chi_{Pc} = [x, y, z]^T$$

CYLINDRICAL (r, θ, z)



$$\chi_{Ps} = [r, \theta, z]^T$$

SPHERICAL (r, θ, ϕ)



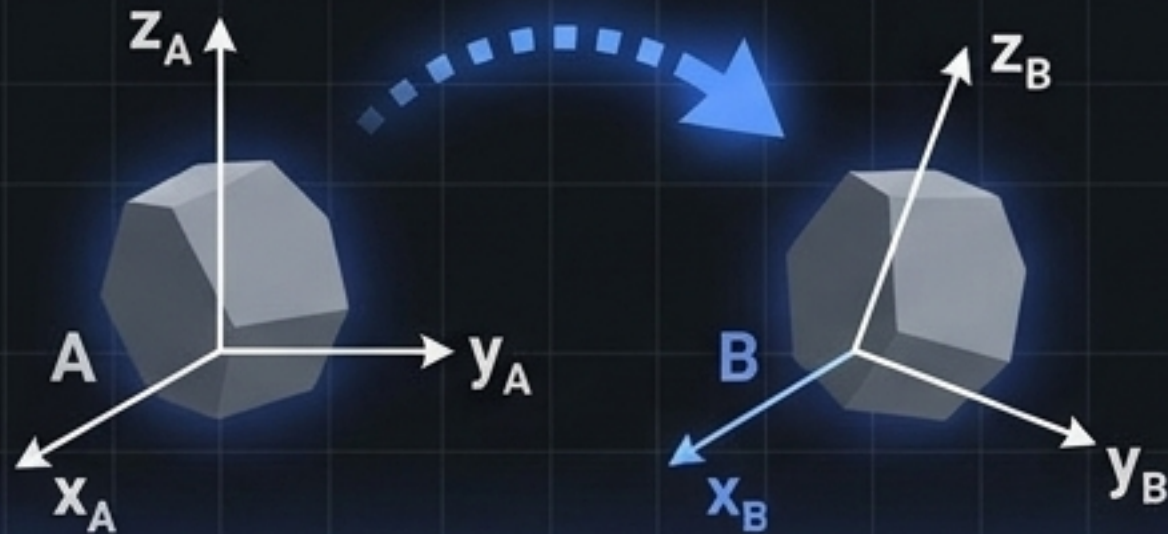
$$\chi_{Pp} = [r, \theta, \phi]^T$$

Tracking an end-effector's position in 3D space requires exactly 3 independent parameters.

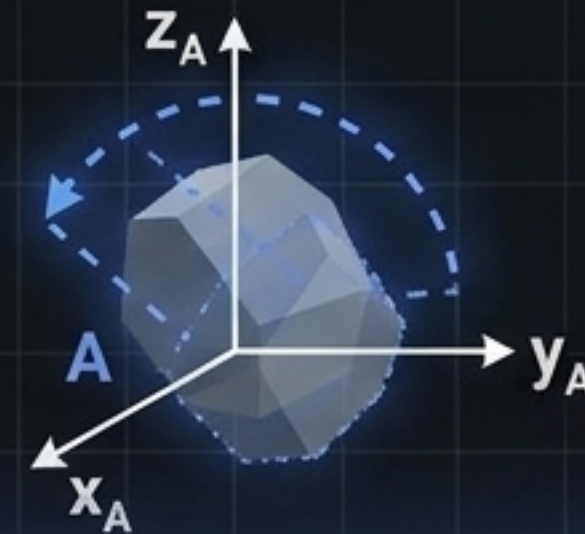
07 THE ORIENTATION PROBLEM

The Complexity of Rotation

PASSIVE ROTATION: Rotating Frame



ACTIVE ROTATION: Rotating Object



Position is defined by 3 independent parameters.

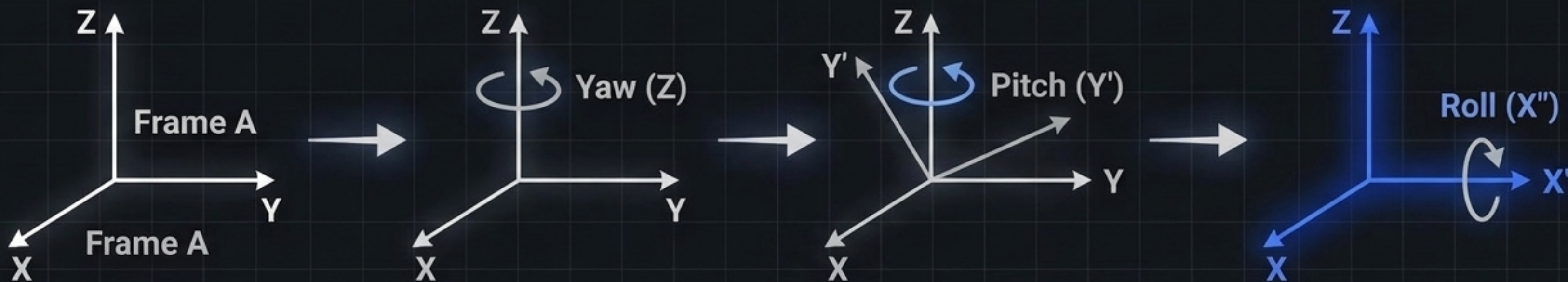
Rotation matrices C_{AB} belong to $SO(3)$. They require 9 parameters, tightly bound by 6 constraints:

$$C_{AB}^T \cdot C_{AB} = I_3$$

We need more efficient representations than a raw 3×3 matrix to avoid numerical drift and computational waste.

08 SUCCESSIVE ROTATIONS

Euler Angles



ZYZ (Proper): Yaw-Pitch-Yaw.

ZYX (Tait-Bryan): Yaw-Pitch-Roll (Standard for flying vehicles).

$$\mathbf{X}_{R,eulerZYX} = [\mathbf{z}, \mathbf{y}, \mathbf{x}]^T$$

CRITICAL WARNING: Different algebra applies to rotations. Never add, subtract, or simply multiply Euler angles. Rotations in 3D are strictly non-commutative.

09 ARBITRARY AXES

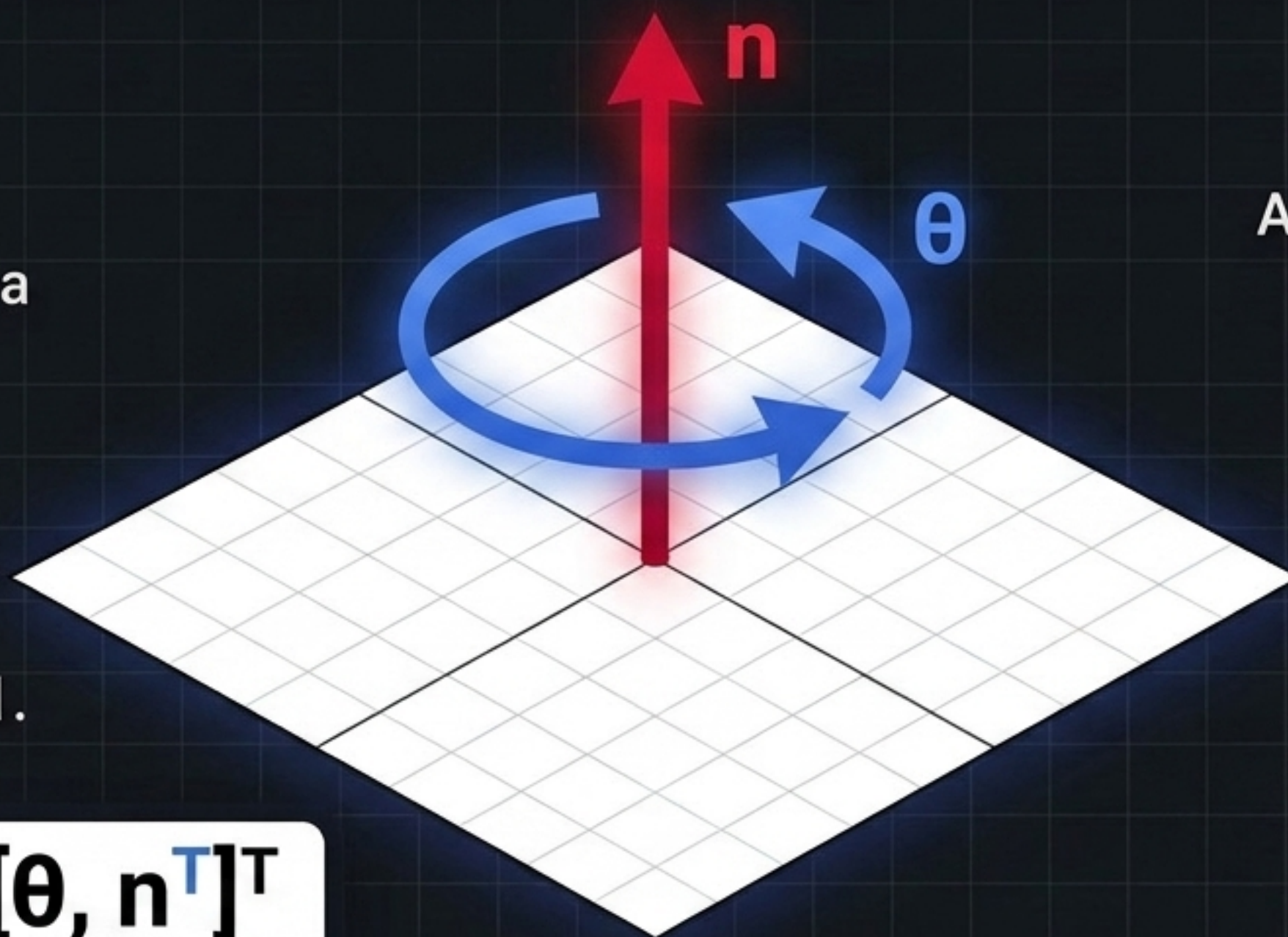
Angle-Axis & Quaternions

ANGLE-AXIS

A non-minimal representation. Define a 3D vector direction 'n' and a scalar rotation magnitude 'θ'.

4 parameters, constrained by $\|n\| = 1$.

$$X_{R, \text{AngleAxis}} = [\theta, n^T]^T$$



QUATERNIONS

An extension of complex numbers. Avoids singularities entirely by embedding rotation in 4D space.

4 parameters:
 $\epsilon_0, \epsilon_1, \epsilon_2, \epsilon_3$.

Standard in modern robotics and graphics.

10 COMPARING ROTATIONS

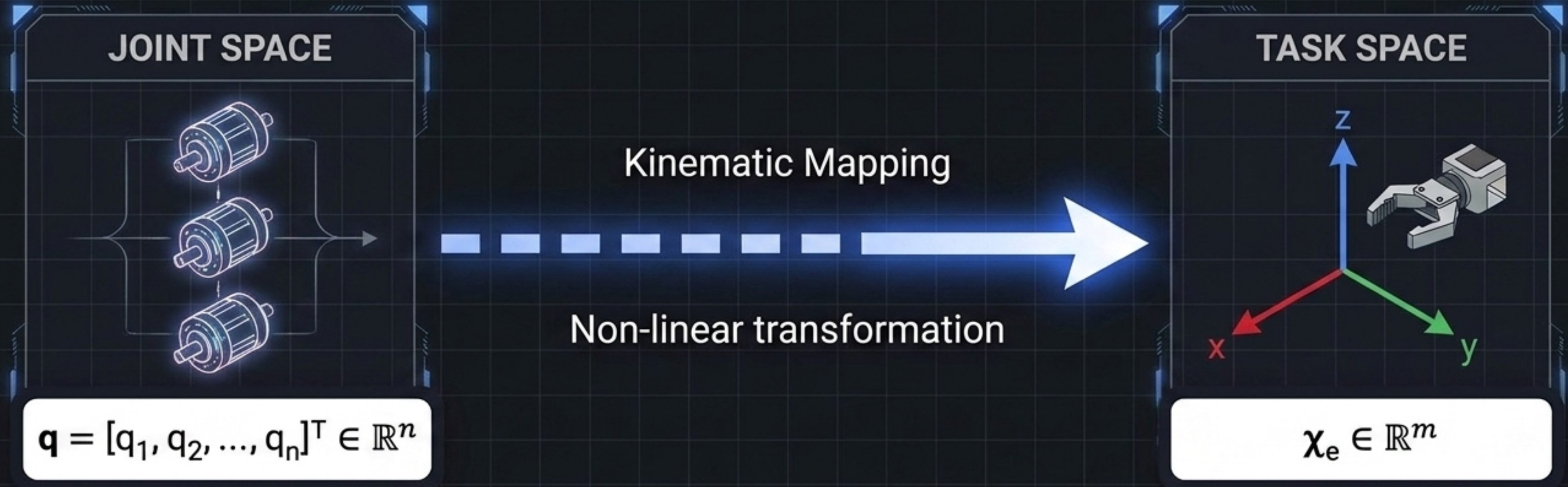
Representation Synthesis

REPRESENTATION	PARAMETERS	CONSTRAINTS	SINGULARITIES (GIMBAL LOCK)	COMPUTATIONAL COST
Rotation Matrix	9	6	None	High
Euler Angles	3	0	YES (Gimbal Lock)	Low
Angle-Axis	4	1	Yes (at $\theta = 0, \pi$)	Medium
Quaternions	4	1	NONE	Highly Efficient

Quaternions provide the optimal balance: avoiding Gimbal Lock entirely without the massive 9-parameter overhead of a full rotation matrix.

11 THE ULTIMATE PROBLEM

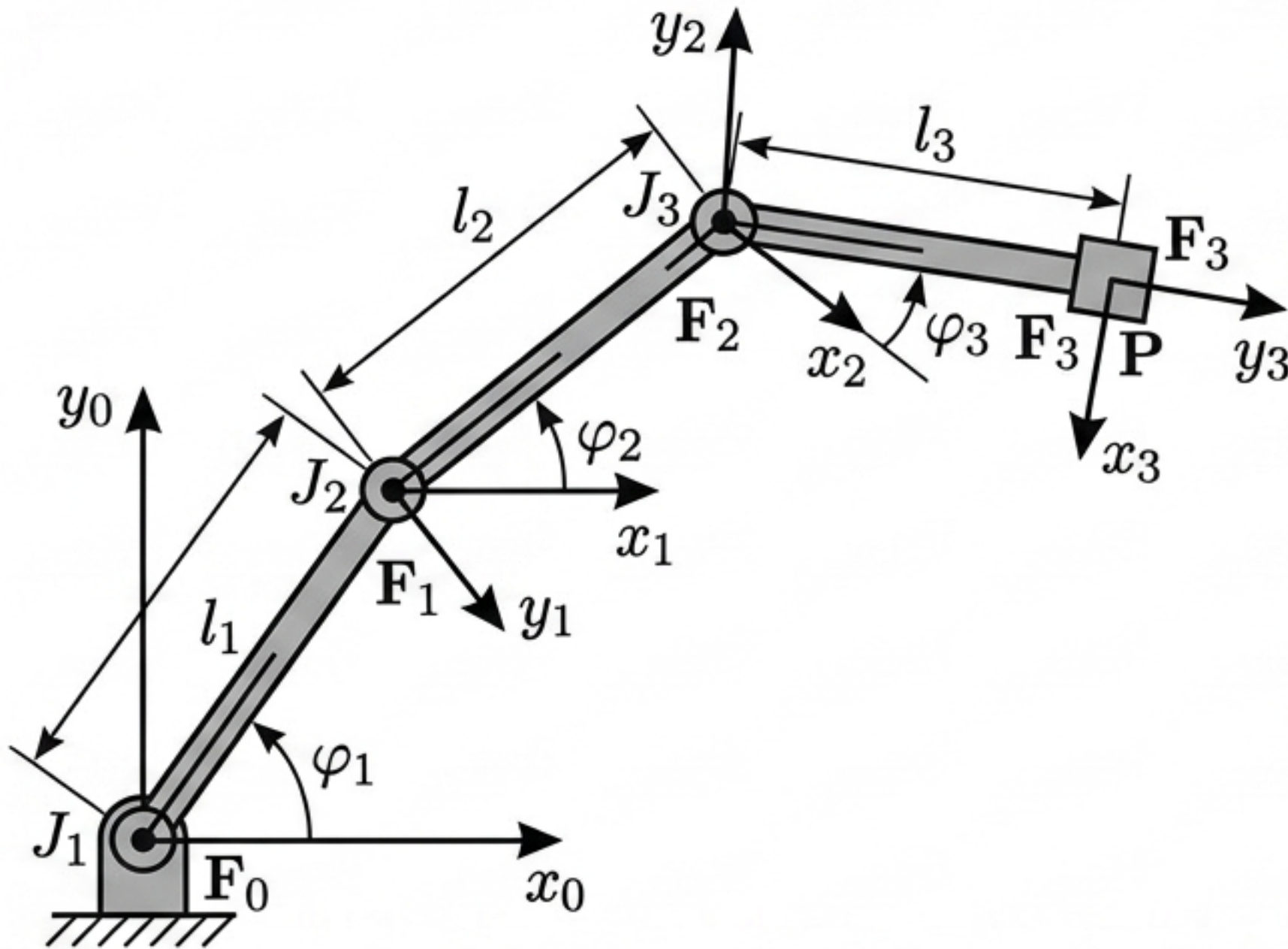
Forward Kinematics Formulation



Given input joint coordinates $\mathbf{q}$, compute the exact position and orientation of the end-effector $\mathbf{x}_e(\mathbf{q})$ in global space.

12 PHYSICAL SYSTEM

3DOF Planar Robot Arm



GENERALIZED COORDINATES:

$$\mathbf{q} = [q_1, q_2, q_3]^T = [\phi_1, \phi_2, \phi_3]^T$$

OBJECTIVE:

Find the end-effector configuration $\chi_e(\mathbf{q})$, broken down into two distinct operational components:

1. Position: $\chi_{eP}(\mathbf{q})$
2. Rotation: $\chi_{eR}(\mathbf{q})$

13 MAPPING SPACE

Solving the 3DOF Arm

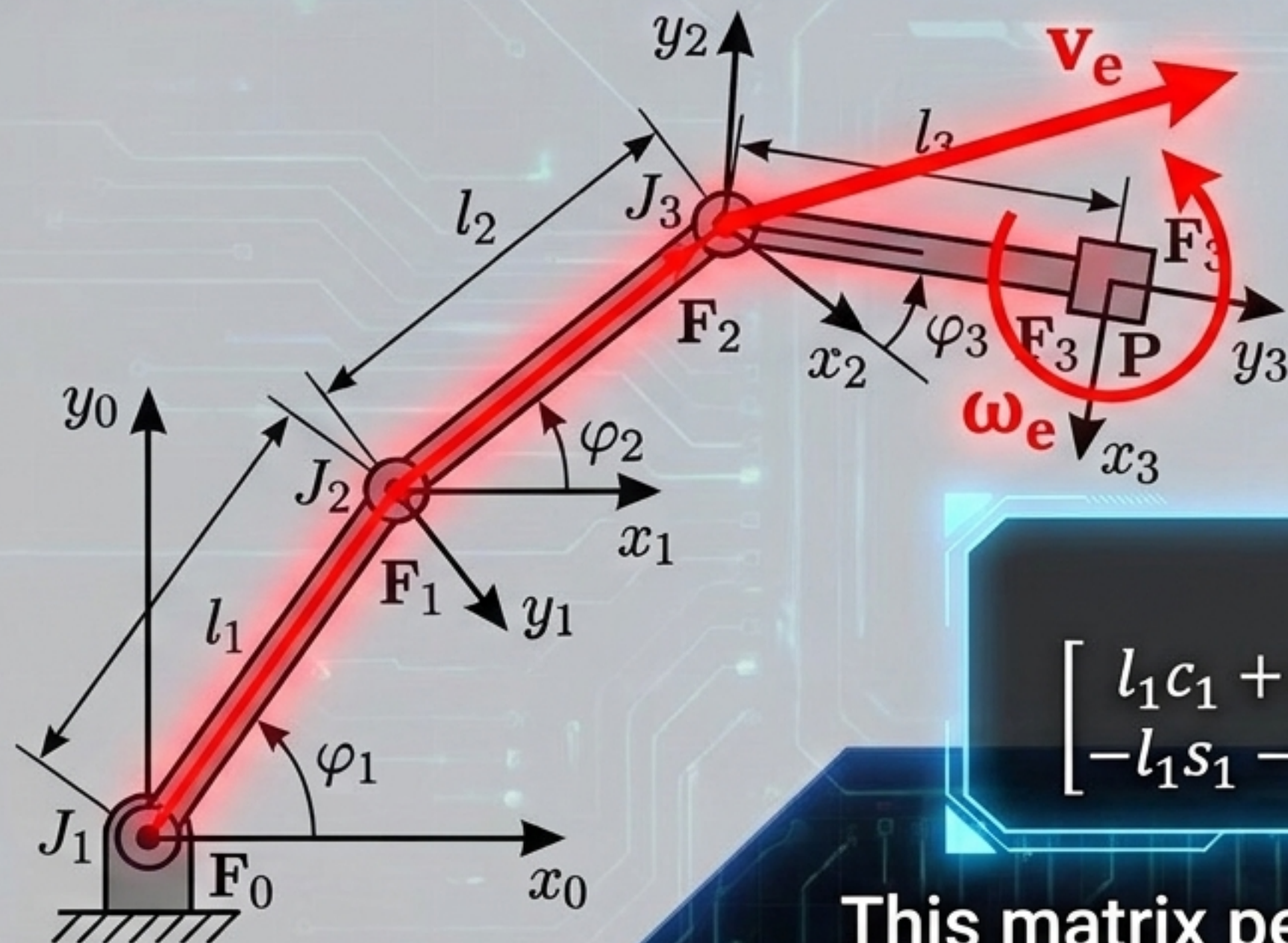
$$\chi_{eP}(\mathbf{q}) = \begin{bmatrix} x \\ z \end{bmatrix} = \begin{bmatrix} l_1 \sin(q_1) + l_2 \sin(q_1 + q_2) + l_3 \sin(q_1 + q_2 + q_3) \\ l_0 + l_1 \cos(q_1) + l_2 \cos(q_1 + q_2) + l_3 \cos(q_1 + q_2 + q_3) \end{bmatrix}$$

$$\chi_{eR}(\mathbf{q}) = q_1 + q_2 + q_3$$

We successfully translated 3 abstract joint angles into a strict Cartesian position (x, z) and absolute orientation in the physical plane.

14 MAPPING VELOCITY

The Jacobian Applied



By taking the partial derivatives of our Forward Kinematics equations with respect to the generalized coordinates, we derive the **Analytical Jacobian**.

$$\mathbf{J}_{eAP}(\mathbf{q}) = \begin{bmatrix} l_1 c_1 + l_2 c_{12} + l_3 c_{123} & l_2 c_{12} + l_3 c_{123} & l_3 c_{123} \\ -l_1 s_1 - l_2 s_{12} - l_3 s_{123} & -l_2 s_{12} - l_3 s_{123} & -l_3 s_{123} \end{bmatrix}$$

This matrix perfectly maps joint velocities to end-effector velocities in Cartesian space: $\dot{\mathbf{x}}_e = \mathbf{J}_{eA}(\mathbf{q})\dot{\mathbf{q}}$